Professional Training in Cybersecurity I

Module 0 — Session 1

The Virtual Lab: VirtualBox, the Course Machines, and SSH

Mohamed Gebril

Session Objective

By the end you can do each of these — on your own machine, not just on the slides:

Explain — what a virtual machine is
the layer that sits between real hardware and an operating system that thinks it owns it

Choose — a network mode on purpose
what each of the five connects, and which one this course runs on

Describe — the lab before building it
three machines with different jobs, meeting on one network named CyberLab

Connect — to a machine with no screen
what SSH is doing, why a key beats a password, and the config that makes it one word

Build — the whole environment
VirtualBox, all three machines, the network between them, in the order that works

Verify — that it is actually working
the end-to-end check, and what each part of it proves

Session Agenda

Part 1 — What a Virtual Machine Is
the layer between hardware and guest, why the lab is virtual rather than spare hardware, and what it asks of your computer

Part 2 — The Network Modes
what each one connects, the network this course runs on, and what it does and does not protect

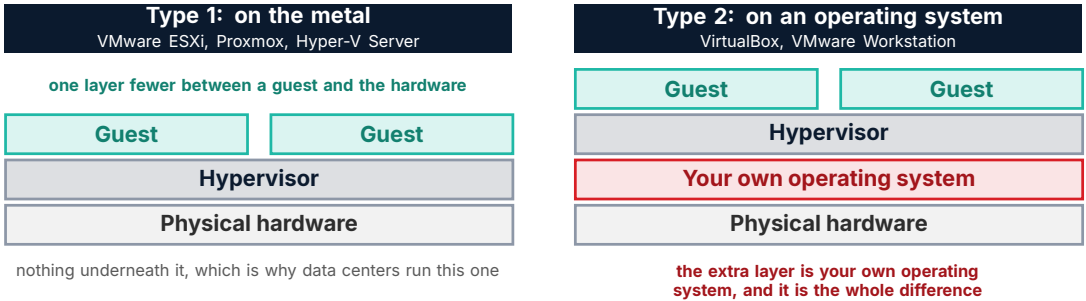
Part 3 — The Three Machines
the workstation, the peer and the target, and what having no screen changes

Part 4 — Reaching the Headless Machine
what SSH is doing, keys instead of passwords, and the client config that makes it one word

Part 5 — Building It
the build order and what forces it, the settings that matter, and what working looks like

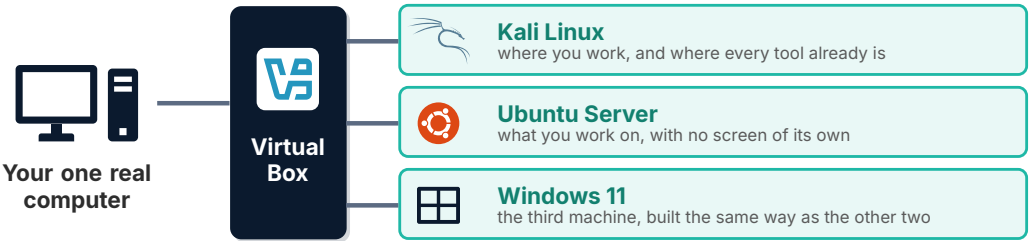
What a Virtual Machine Is

Type 1 and Type 2: Where the Hypervisor Sits



- **A hypervisor either owns the hardware or borrows it, and that single choice is the whole classification.**
- Type 1 is installed onto bare metal with no operating system beneath it. Every cloud server you will ever rent is running on one.
- Type 2 installs like any other program, on the operating system you already use. It is slower, and it is the only kind you can run on a laptop you also need for everything else.
- **VirtualBox is Type 2**, which is why the lab costs you an install rather than a spare computer.

One Machine, Many Machines: What a Hypervisor Does



- **A virtual machine is a whole computer running as a program on the computer you already own.**
- VirtualBox hands each guest a slice of the real processor, memory and disk, and convinces it that the slice is a machine of its own.
- The guests cannot see each other unless you connect them, which is what makes one safe to break while another keeps working.
- **Three machines, and all three get built.** Kali and Ubuntu Server today, Windows 11 the same way, so the lab is complete before the first module that needs any of them.

Why the Lab Is Virtual, Not Spare Hardware

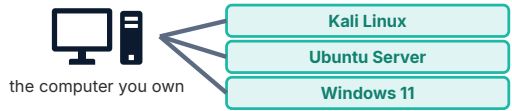
If the lab were real hardware



three computers to buy, power and store, and the cable between them

a machine you break stays broken until it is rebuilt

The lab as it is actually built

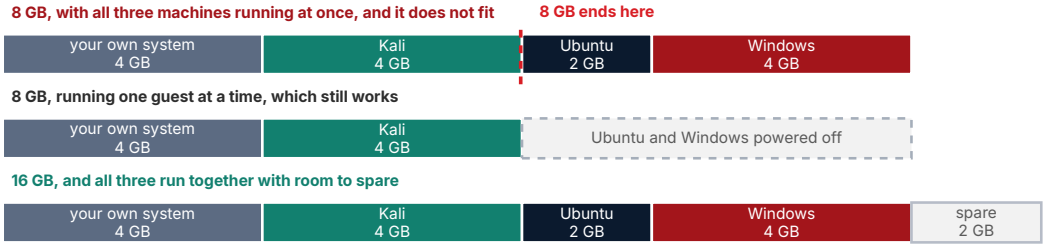


one machine, and the network between them is a setting

a machine you break is restored from a snapshot in seconds

- **The same three machines, and the difference is what they cost and whether you can put them back.**
- In hardware this lab is three computers, a switch, and the cables between them. Built virtually it is disk space, and the network is a setting rather than a purchase.
- A machine you break in hardware stays broken. A machine you break here is restored from a snapshot in seconds, which is what makes it safe to run a tool you do not yet fully understand.
- It also makes every lab in the room identical, so a failure on your machine is a failure on mine, and a failure that reproduces is one that can be fixed.

What VirtualBox Needs From the Host Machine



- **With Windows in the lab, 16 GB is the requirement rather than the recommendation.** The top row runs past what an 8 GB machine has before it reaches the end.
- Eight gigabytes still works if you run one guest at a time, and that is a real way to take this course. It is slower, because several exercises need two machines talking to each other.
- Disk moves with it. Windows wants roughly 40 GB of its own, so the floor is about 100 GB free rather than 60.
- **Intel VT-x or AMD-V is not a number but a switch.** It lives in your firmware settings, it is off by default on many machines, and it cannot be turned on from inside the operating system.

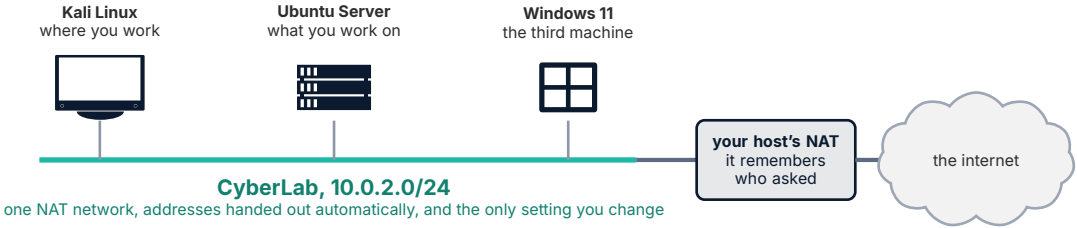
The Network Modes

Network Modes: Where VirtualBox and VMware Differ

Mode	VM to VM	VM to host	VM to internet	Typical use
VirtualBox (the taught path)				
Internal Network	**Yes**	–	–	Sealed-off testing
Host-only	**Yes**	**Yes**	–	Private lab, no internet
NAT	–	**Out only**	**Out only**	One VM with internet
NAT Network	**Yes**	**Out only**	**Out only**	**This course: all three VMs**
Bridged	**Yes**	**Yes**	**Yes**	Visible to your whole LAN
VMware Workstation (reference only, for a student who already runs it)				
LAN Segment	**Yes**	–	–	Sealed-off testing
Host-only (VMnet1)	**Yes**	**Yes**	–	Private lab, no internet
NAT (VMnet8)	**Yes**	**Yes**	**Out only**	Several VMs with internet
Bridged (VMnet0)	**Yes**	**Yes**	**Yes**	Visible to your whole LAN

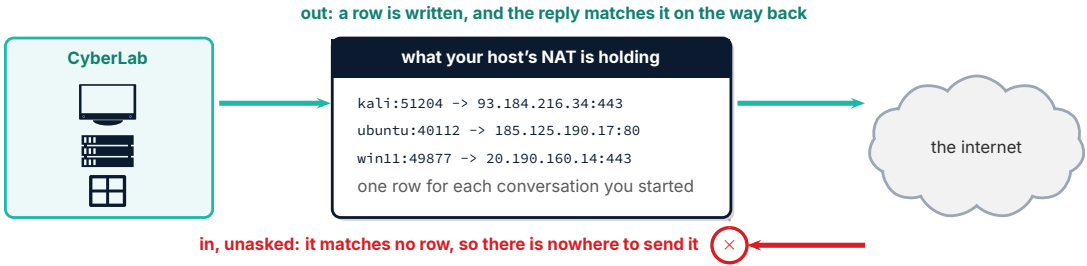
Out only: the VM can start a connection, but nothing can start one to it unless you open a port forward on purpose. A dash means no path at all.

The Lab’s Own Network: CyberLab



- **All three machines sit on one network named CyberLab, and the adapter setting is the only thing you change.**
- They can see each other because they share it. That is what lets a tool running on Kali reach the server, or the Windows machine, at all.
- They reach the internet through your own computer, which is how each of them installs updates and packages.
- Addresses are handed out automatically on 10.0.2.0/24, so you read a machine’s address off the machine rather than choosing it.

What CyberLab Blocks, and How It Knows



- **Nothing outside can start a conversation with your machines, and no rule was written to stop it.**
- NAT writes a row every time one of your machines reaches out, and it uses those rows to send the replies back to the right machine.
- An arrival nobody asked for matches no row, so NAT has no machine to give it to and it is dropped. Protection here is a side effect of how the translation works.
- **None of that constrains what leaves.** Outbound rows are written for whatever you aim at, so the outward boundary is the authorization rule and not this table.

The Three Machines

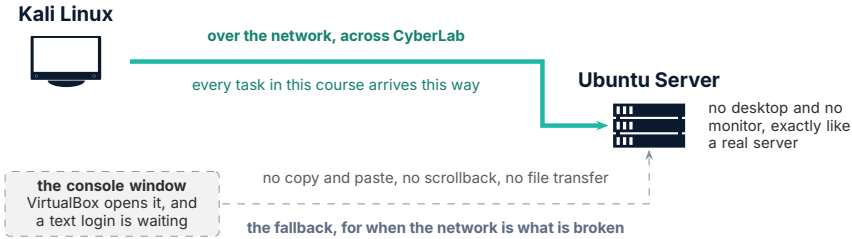
Three Machines, and Three Different Jobs

Kali Linux	Ubuntu Server	Windows 11
		
a desktop, with the tools installed	a text console, almost nothing else	a desktop, like an office machine
you sit at it, at its own screen	over the network, from Kali	at its screen, or over the network
the machine you work from	the peer you practice against	the target later modules land on

All three are disposable. A snapshot taken first restores any of them exactly as it was, which is what makes breaking one safe.

- **Three machines with three jobs. One holds the tools, one is the peer you practice against, and one is what later modules attack.**
- Kali Linux arrives built, with a desktop and the tool set already installed, so there is nothing to configure before the first exercise.
- Ubuntu Server arrives nearly empty, and that is deliberate. Every service it runs later is one you install on purpose, which is how you learn what a service actually needs.
- Windows 11 is the machine the malware and post-exploitation work lands on, so it is built like an ordinary office computer rather than hardened or stripped.

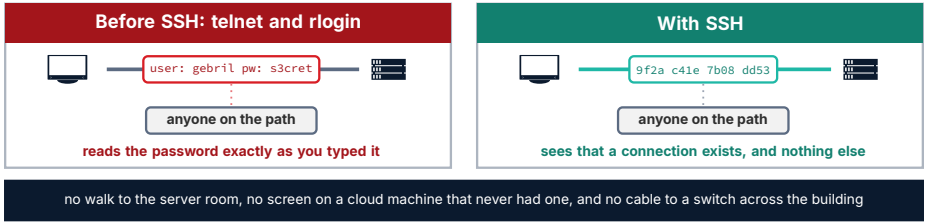
Why the Server's Console Is Not Where Work Happens



- **The server has no desktop, so the network is not one way in. It is the way in.**
- VirtualBox does open a console window for it, and a text login is waiting there. It works, and it has no copy and paste, no scrollback, and no way to move a file.
- That makes the console a fallback, kept for the case where the network itself is what is broken. Everything else arrives over the network, and the next part is what that actually means.
- **Kali and Windows are not like this.** Both have a desktop you can sit at, and Ubuntu Server is the one machine in the lab you reach only over the wire, exactly as real servers are run from another building entirely.

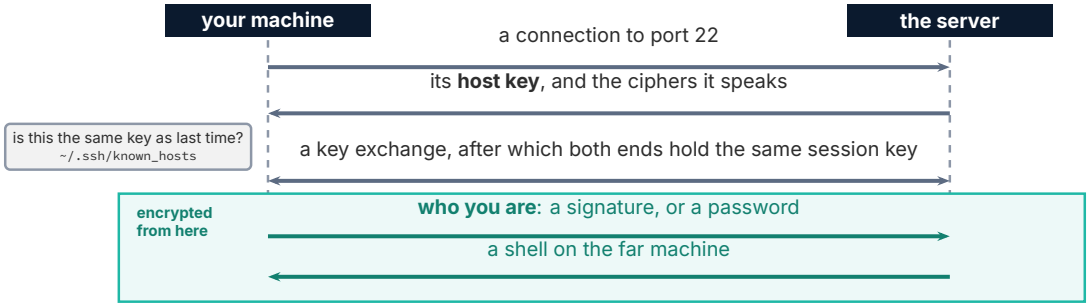
Reaching the Headless Machine

What SSH Is, and Why Every Server Uses It



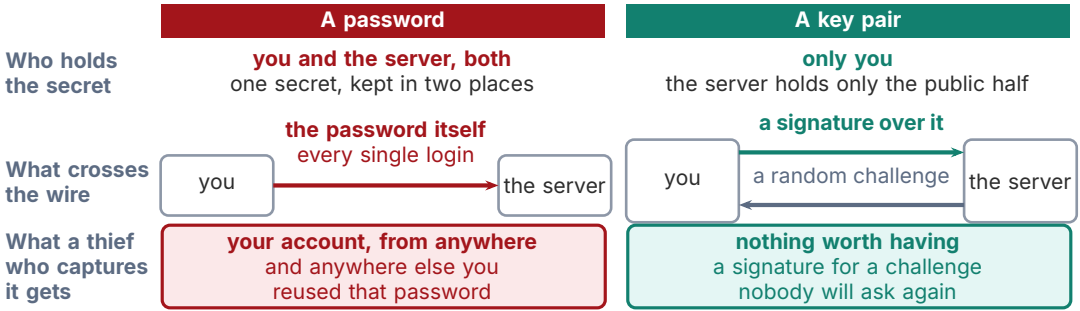
- **SSH gives you a shell on another machine, and encrypts everything that shell carries.** The second half is why it replaced what came before.
- Telnet and rlogin did the same job in clear text. A password typed into telnet crossed the network readable by anyone between the two machines, and that was ordinary practice until it stopped being survivable.
- It is not only a shell. The same connection carries file copies, forwarded ports, and remote editing, so one channel does the work of several tools.
- **This is how servers are actually administered.** A machine in a rack downstairs, a cloud instance in another country, a switch on the far side of the building: none of them is walked to, and a cloud machine has no screen to walk to in any case.

What SSH Does Before a Password Is Ever Typed



- **The channel is encrypted before anyone proves anything**, which is why nothing typed into SSH crosses in the clear.
- **The server proves itself first.** Your client compares its host key against `~/.ssh/known_hosts`, and the first connection has nothing to compare against, so it prints a fingerprint and asks. That one prompt is where the trust starts.
- If that key later changes, the client refuses loudly rather than connecting quietly.

Passwords and Keys: What Each Sends, and What It Costs



- **Both methods run inside the encrypted channel, so neither one is sent in the clear.** What differs is what is sent at all.
- A key pair is made once with `ssh-keygen -t ed25519`. The public half is copied to the server with `ssh-copy-id`, and **the private half never leaves your machine**.
- Passwords are not obsolete and you will use them constantly. Keys are what a server exposed to the internet should accept, which is why every later session uses them.

The Client Config That Turns a Command Into One Word

~/.ssh/config

```
Host lab                                # the one word you will type
    HostName 10.0.2.6                   # where to connect
    User student                        # who to log in as
    IdentityFile ~/.ssh/id_ed25519      # which key to offer
```

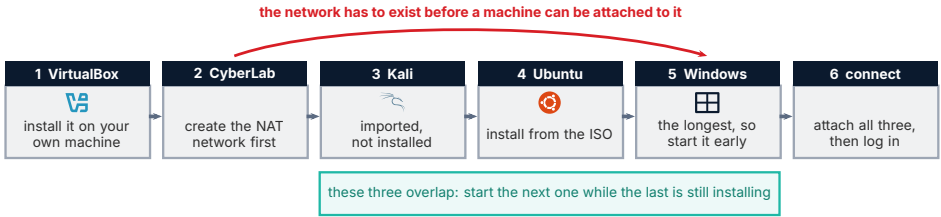
```
$ ssh lab
student@ubuntu:~$
```

no address, no account, no key path, and nothing to remember

- **One file turns a command nobody remembers into a word everybody does.** It lives on your own machine, not on the server.
- Each `Host` block names an alias and the settings behind it. After that, `ssh lab` supplies the address, the account, and the key without any of them being typed.
- The same file is read by `scp`, `sftp` and `rsync`, so sending a file to the server becomes `scp report.md lab:` using that one alias.
- When the server's address changes, one line in this file changes with it. That is the real reason to write it down instead of retyping the command.

Building It

The Build Order, and What Forces It



- **Create the network before the machines.** It is the one step that cannot be done later without going back and editing every machine you already made.
- Kali is imported rather than installed, so it is running within minutes. Ubuntu and Windows are real installs and Windows is much the longest, which is why it starts early.
- **We start all three together in the session, and whatever is not finished you finish at home.** The guide is written to be followed without help from wherever you stopped.
- **Windows is installed with no product key**, using the option that says you do not have one. It runs indefinitely that way, with a watermark in the corner and personalization locked, and neither affects a single exercise.

The Settings That Matter, and What Breaks

The setting	What it must be	What breaks if it is wrong
Before anything boots		
Virtualization in your firmware	**Intel VT-x or AMD-V, enabled**	64-bit guests vanish from the creation wizard, or the machine will not start
When CyberLab is created		
The network's name and range	**CyberLab, 10.0.2.0/24**	the machines sit on different networks and cannot see each other at all
Its DHCP server	**on, which is the default**	no machine is given an address, so every check later in this session fails
During each install		
OpenSSH server, in the Ubuntu installer	**checked. The expensive one to miss**	no way into the server over the network, so you install it by hand at the console
Once it works		
A snapshot of each machine	**before the first exercise**	a broken machine costs a rebuild instead of a restore, an evening instead of a minute

What Working Looks Like, and How to Check

kali@kali:~$ on the workstation, in this order

```
$ ip a | grep 10.0.2                  # 1  do I have an address
  inet 10.0.2.5/24 brd 10.0.2.255 scope global eth0
$ ping -c 1 10.0.2.6                  # 2  can the two machines see each other
  64 bytes from 10.0.2.6: icmp_seq=1 ttl=64 time=0.42 ms
$ ssh student@10.0.2.6                # 3  the way in
  student@ubuntu:~$
```

Check 2 will fail against the Windows machine. Its firewall drops unsolicited pings by default, so that one failure is correct behavior and not a broken lab.

- **Each check depends on the one before it, so the first failure tells you where the lab actually stops.**
- An address proves the machine joined CyberLab. A reply proves the two machines share it. A shell proves the server is reachable the way every later session will use it.
- Run all three from the workstation. The addresses shown are worked examples and yours come from the first check. If something fails, note the exact step and the exact error and bring both.

Snapshots: When to Take One, and What to Name It



Two is usually enough per machine. Each one holds a copy of the disk as it was, so a pile of them fills your disk.

- **Take one the moment a machine first works, and take one before anything you expect to break it.** Those two cover almost everything.
- **Name it for the state it holds**, not for the date. *Clean install* and *before the web exercise* tell you what you are going back to. A date does not.
- Restoring is not undoing the last thing you did. It puts the whole machine back to that point, so anything you wanted to keep comes out first.
- They are not free. Each one keeps the disk as it was at that moment, so two per machine is usually right and ten is a full disk.

Key Takeaways

- A virtual machine is **a whole computer living in a file on your own**, which is why this lab is disposable and a lab built on spare hardware is not.
- **A network mode is a decision about who can reach whom**, made on purpose. CyberLab puts the three machines on a network of their own.
- **Isolation is not symmetric.** Nothing outside can start a conversation with your machines. Nothing stops your machines from starting one outward, and that half is the authorization rule.
- **A machine with no screen is the normal case in this field**, and SSH is how it is used. A key beats a password because what crosses is a signature rather than the secret.
- **The build order is forced, not preferred:** the network exists before a machine can attach to it, and the checks run in the same order.
- **Take a snapshot the moment a machine first works.** That is the difference between a restore and a rebuild.

How the Practical Guide Uses This

1. **The Practical Guide is what you build from.** The same machines in the same order, with the exact settings and the exact commands rather than the reasons behind them. Those were today.
2. **We start all three machines together, and you finish what is left at home.** It is written to be picked up from wherever you stopped, by somebody working alone.
3. **The Submission worksheet is filled in as you go.** Every build step names what to capture, and it is one screenshot or one line of output taken at the moment it works.
4. **The Home Task finishes the lab and proves it**, using those same checks from each machine.

Two documents sit beside you while you work rather than being read through: the **Tooling Reference** covers every tool used here in full, and the **Troubleshooting FAQ** is organized by the symptom in front of you.

Open the Practical Guide

`Module0_Session1_Practical_Guide.md`

Module 0 — Session 1
The Virtual Lab: VirtualBox, the Course Machines, and SSH